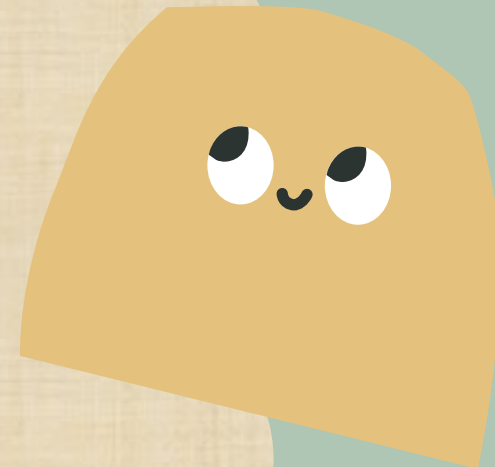
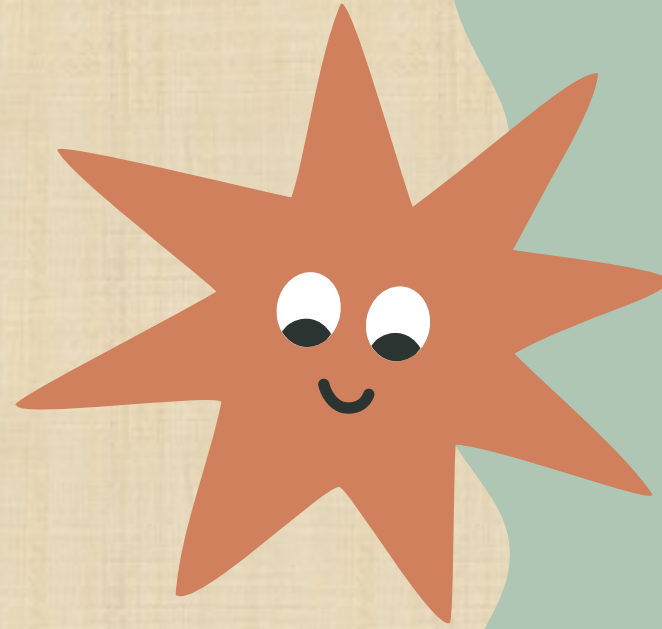


UNIT 2 BASICS OF PROGRAMMING OF C++

10TH COMPUTER SCIENCE
SINDH TEXTBOOK BOARD

<https://awinashgoswami.github.io>



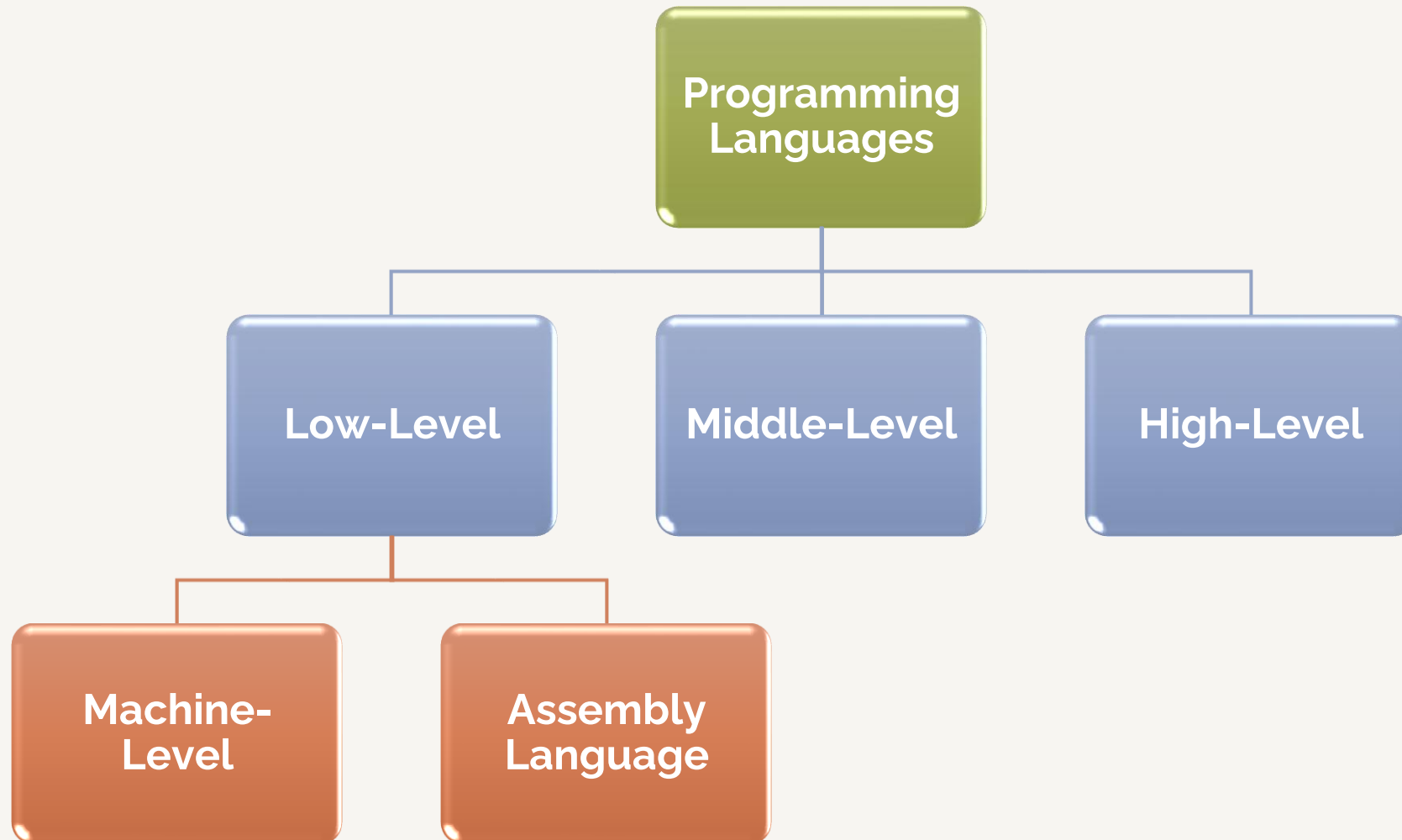
Agenda

- Introduction
- Programming languages
- Translators

Introduction

- A computer program (also known as software) is a set of instructions that perform a specific task. E.g. MS Word (Creating Documents), CapCut (Video Editing), VS Code (Coding) etc.
- Different languages are used to write these software such as; C++, Java, Python, C# etc.
- Syntax is like a 'Grammar of Programming' that follows programming rules. For example: `cout<<"Hello World";`
- The above C++ coding example follow rules to print text – Hello World. If we remove semicolon, it rules out programming grammar which can throw an error.

Classification of Programming Languages



Low-Level Programming Language

- Low Level Language can directly communicate with the hardware. Types of Low-Level Language are Machine level language and Assembly language.
- *Machine-level language:*
 - The machine-level language has direct access to the hardware.
 - It cannot be easily understood by humans.
 - The machine-level language is written in binary digits, i.e., 0 and 1.
 - It does not require any translator as the machine code is directly executed by the computer.
- *The assembly language:*
 - The assembly language comes above the machine language means that it has lesser access to hardware.
 - It is easy to read, write, and maintain by humans.
 - The assembly language is written in simple English language, so it is easily understandable by the users.
 - In assembly language, the assembler is required to convert the assembly code into machine code.

Middle-Level Programming Language

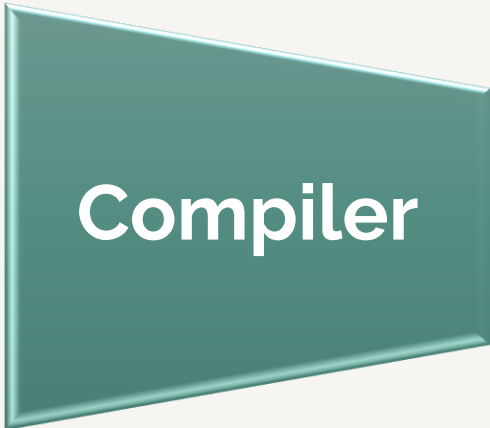
- A middle-level programming language combines the low-level control of hardware with the readable syntax of high-level languages.
- It allows direct memory manipulation since it is near to hardware.
- Prominent examples are C, C++, and Rust.

High-Level Programming Language

- A high-level programming languages are closer to human languages than machine-level languages (0,1)
- They don't have direct access to hardware therefore; a translator is required to translate a high-level language into a low-level language.
- These languages are portable and independent of machine.
- Example of high-level languages are Python, Java, JavaScript etc.

Translators

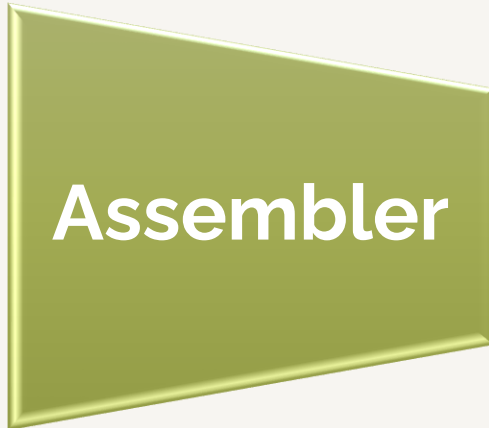
- Computers only understand binary language. Human write code in high-level language.
- The code written is called source code.
- In order to build a communication between the code written by human and understand by machine, we need translator.
- There are three different types of translators given below.



Compiler



Interpreter



Assembler

Compiler

- A compiler is used to convert high-level programming language to low-level programming language all at once and creates an executable file.
- This translated program can be used again and again without the need for recompilation from source code.
- If the program fails in conversion, it throws errors and may cause program to crash.
- These errors can only be fixed by changing the original source code and compiling the program again.
- Examples of compilers; GCC / Clang (C++), Javac (Java), tsc (typescript compiler)

Interpreter

- Interpreter is also a translator used to convert high-level programming language to low-level programming language.
- However, interpreter translates the code line by line and reports the error as soon as it is encountered during the translation process.
- With interpreter, it is easier to detect errors in source code than in a compiler.
- An interpreter is faster than a compiler as it immediately executes the code upon reading the code.
- Interpreters do not create an executable file.
- Examples of Interpreters; CPython & PyPy, JavaScript Interpreter, PHP Interpreter

Assembler

- An assembler is a translator used to translate assembly language to machine language.
- It is like a compiler for the assembly language but interactive like an interpreter.
- Examples of assemblers; Netwide Assembler, GNU Assembler

Compiler VS Interpreter VS Assembler

Feature	Compiler	Interpreter	Assembler
Definition	Translates high-level language to machine code	Translates high-level language to machine code	Translates assembly language to machine code
Input Language	High-level language (C, Java, C++)	High-level language (Python, JavaScript)	Assembly language (MOV, ADD, JMP)
Translation Method	Whole program at once	Line by line	Instruction by instruction
Error Reporting	After complete translation (all errors together)	Immediately when error is found (line by line)	Immediately when error is found (instruction by instruction)
Speed of Translation	Slow (takes time to compile)	Fast (starts executing immediately)	Fast (simple 1-to-1 translation)
Examples	GCC, Clang, javac	Python IDLE, Node.js	NASM, MASM, GAS
Best For	Large programs where speed matters	Small programs, scripting, debugging	Low-level hardware programming
Analogy	Reads whole book, then summarizes	Reads and explains each line instantly	Translates each word one by one

Types of Errors

- Errors are the problems that occur in the program which cause the program to behave abnormally.
- Programming errors often remain undetected until the program is compiled or executed.
- Most common programming errors can be classified in following.



**Syntax
Error**

**Run-Time
Error**

**Logic
Error**

Syntax Error

- Syntax error occurs when the code does not follow the rules of the programming language.
- These can be mistakes such as misspelled keywords, a missing punctuation character, a missing bracket, or a missing closing parenthesis.
- Integrated Development Environments (IDEs) detect these errors as you type and underline the faulty statements with a wavy line.
- If you try to execute a program that includes syntax errors, you will get error messages on your screen and the program will not be executed.
- Most frequent syntax errors are:
 - Missing Parenthesis ()
 - Printing the value of variable without declaring it
 - Missing semicolon

Run-Time Error

- Errors which occur during program execution (run-time) after successful compilation are called run-time errors.
- A run-time error occurs when a program is asked to do something that it cannot perform, resulting in a 'crash'.
- The widely used example of a run time error is asking a program to divide by 0.
- The code contains no syntax or logic errors but when it runs it can't perform the task that it has been programmed to carry out.

Logic Error

- Logic errors are those errors that prevent your program from doing what you expected it to do.
- These types of errors which provide incorrect output but appear to be error free are called logical errors.
- With logic errors you get no warning at all.
- For example, consider a program that prompts the user to enter three numbers, and then calculates and displays their average value. The programmer, however, made a logic error; one of its statements divides the sum of the three numbers by 5, and not by 3 as it should. The program will execute as usual, without any error messages, prompting the user to enter three numbers and displaying a result, but not the correct one. This one is example of logical error.